

Design and Architecture

P2P Car Rental System

Course	MSE800 Professional Software Engineering
Assignment	Assignment 1 - Object-Oriented Programming
Task	Task 1 - Design and Architecture (30%)
Document	Design and Architecture
Version	1.0.0
Date	May 2026

This document presents the complete object-oriented design and system architecture of the P2P Car Rental System, including ERD, Use Case, Class, Sequence, and Activity diagrams along with detailed architectural explanations.

Table of Contents

Section	Title	Page
1	System Overview	3
2	Architectural Style and Layered Design	4
3	Entity Relationship Diagram (ERD)	6
4	Use Case Diagram	7
5	Class Diagram	8
6	Sequence Diagram - Make Booking	9
7	Activity Diagrams	10
7.1	Activity: Make Booking	10
7.2	Activity: Add Car Listing	11
7.3	Activity: Approve / Reject Booking	12
7.4	Activity: Handle Insurance Claim	13
8	Design Patterns Applied	14
9	Component Interactions Summary	15

1. System Overview

The **P2P Car Rental System** is a peer-to-peer marketplace platform that connects two distinct user groups: **car owners** who want to monetize their idle vehicles, and **renters** who need short-term access to a car. A third actor, the **admin**, moderates the platform to ensure trust and safety.

Unlike traditional car rental companies that own a fleet, this platform owns no vehicles — it earns revenue through a 15% commission on every transaction. This is the same model used by Airbnb (homes), Uber (rides), and Turo (cars in the US).

1.1 Three Actors and Their Roles

Actor	Primary Role	Key Use Cases
Owner	Lists cars on the platform and earns rental income.	Add/Remove car listing, Approve/Reject booking, View earnings.
Renter	Searches and books cars from available listings.	Search & filter cars, View details, Make booking, Cancel booking, Submit review.
Admin	Moderates the platform — verifies users, approves cars, handles disputes.	Verify users, Approve/Reject cars, View bookings, Handle insurance claims, View reports.

1.2 Technology Stack

Layer	Technology
Language	Python 3.8+
Database	SQLite 3 (with WAL mode for performance)
Interface	Command-Line Interface (CLI)
Libraries	tabulate (pretty tables), colorama (color output)
Architecture	Layered (Presentation / Service / Model / Data Access)

2. Architectural Style and Layered Design

The system follows a classic **4-layered architecture**, a well-established pattern that separates concerns and makes the codebase maintainable.

2.1 The Four Layers

Layer	Folder	Responsibility	Example Files
Presentation	ui/	CLI menus and user interaction. No business logic here.	main_menu.py, owner_menu.py, renter_menu.py, admin_menu.py
Service (Business Logic)	services/	Core business rules: bookings, pricing, authentication.	auth_service.py, booking_service.py, car_service.py
Model (Domain)	models/	OOP entities representing real-world concepts.	user.py, owner.py, renter.py, admin.py, car.py, booking.py
Data Access	database/	SQLite connection management and SQL queries.	db_manager.py (Singleton pattern)

2.2 Why Layered Architecture?

Separation of concerns: Each layer has ONE job. UI code doesn't know about SQL; database code doesn't know about menus.

Replaceability: We could swap the CLI for a web UI without touching the service or data layers. Or swap SQLite for PostgreSQL without touching any business logic.

Testability: Services can be unit-tested independently of the UI. We can mock the database to test business rules in isolation.

Team scaling: Different developers can own different layers without stepping on each other's toes.

Maintainability: When a bug is reported, we know exactly which layer to look in based on the symptom.

2.3 Supporting Modules

Module	Purpose
config.py	Centralized configuration. Supports multiple environments (dev/test/prod) via APP_ENV environment variable.
factories/	Factory pattern for creating User subclasses by role.
patterns/	Strategy pattern for interchangeable insurance plans.
utils/	Reusable helpers — input validators, password hashing, CLI display utilities.

Dependency Rule: A higher layer may call into the layer immediately below it, never the reverse. So *ui* calls *services*, which calls *models* and *database* — but never the opposite. This one-way flow prevents circular dependencies and tight coupling.

3. Entity Relationship Diagram (ERD)

The ERD shows the physical database schema — 5 tables with their columns, primary keys (PK), foreign keys (FK), and the relationships between them.

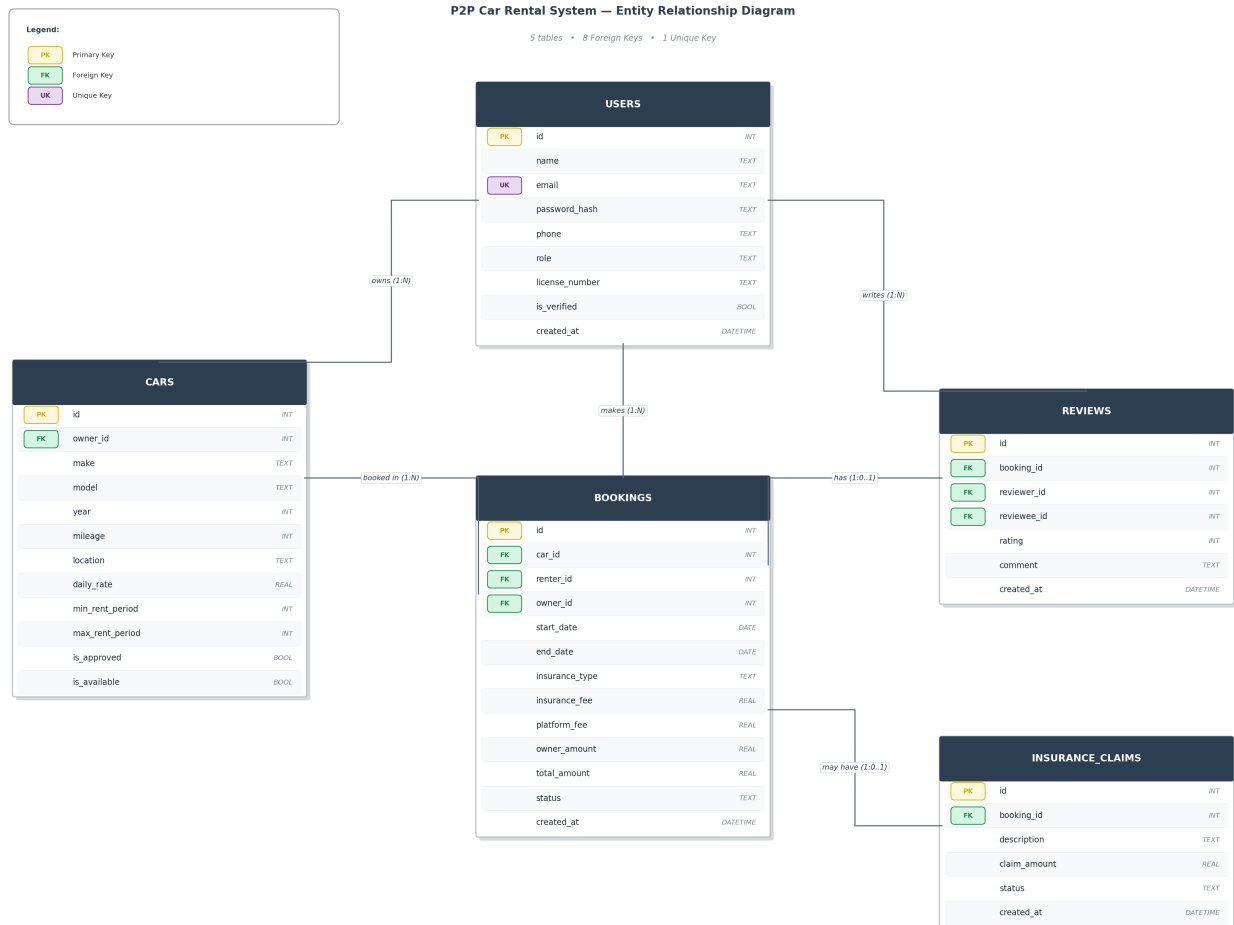


Figure 1: Entity Relationship Diagram showing 5 tables, 8 foreign keys, and 1 unique key.

3.1 Key Relationships

Relationship	Cardinality	Meaning
USERS → CARS	1 : N	One owner can have many cars
USERS → BOOKINGS	1 : N	One renter can make many bookings
CARS → BOOKINGS	1 : N	One car can be in many bookings (over time)
BOOKINGS → REVIEWS	1 : 0..1	Each booking may have one review
BOOKINGS → CLAIMS	1 : 0..1	Each booking may trigger one insurance claim

4. Use Case Diagram

The Use Case diagram captures what each actor can do with the system, and how use cases relate via **«include»** (mandatory sub-tasks) and **«extend»** (optional add-ons).

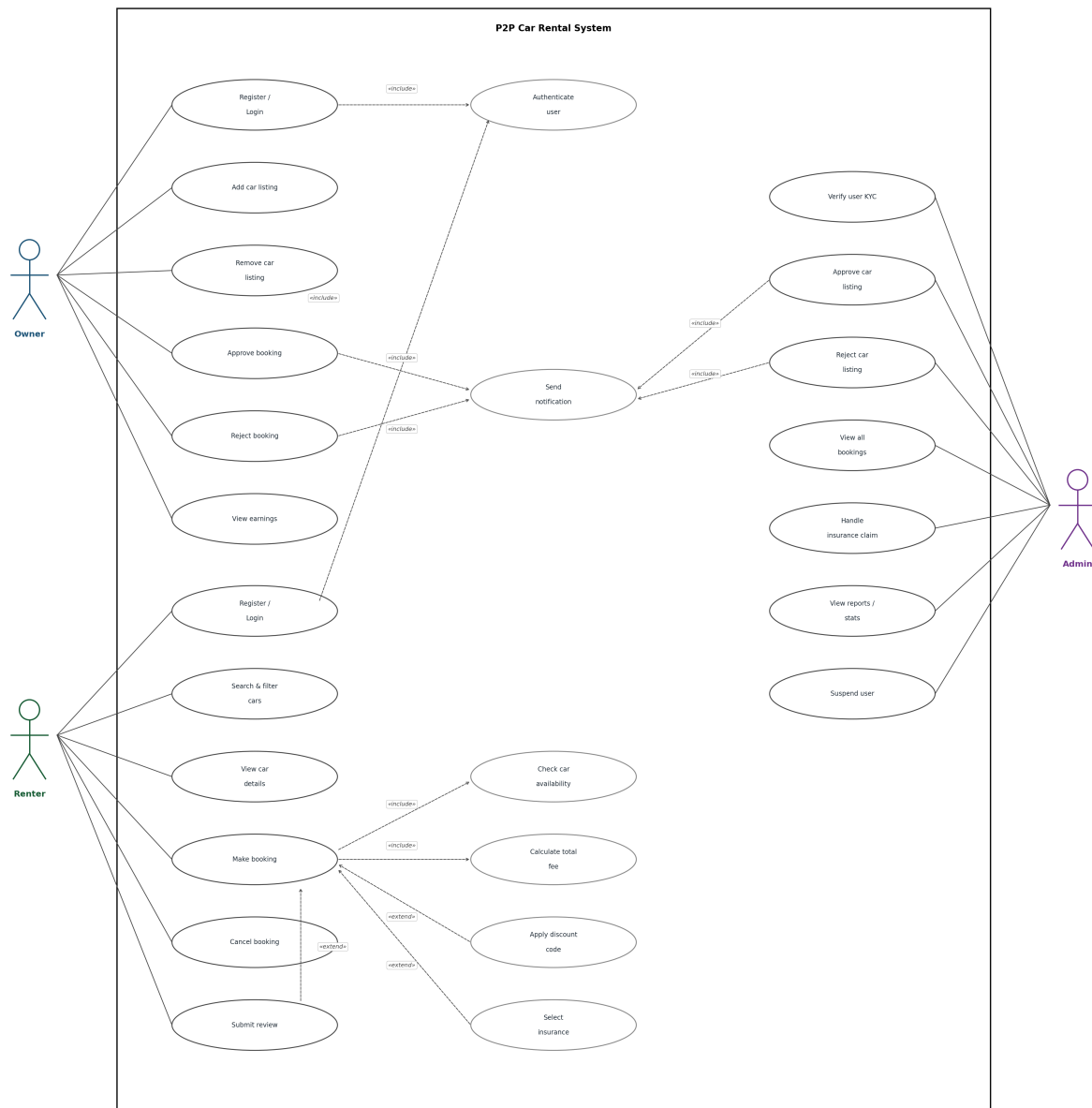


Figure 2: Use Case Diagram — 3 actors, 19 use cases, with «include» and «extend» relationships.

4.1 Notable Relationships

«include» means the base use case ALWAYS calls the included use case as a mandatory sub-task. Examples in our system:

- *Make booking* **«include»** *Check car availability* (must verify before booking)
- *Make booking* **«include»** *Calculate total fee* (price is always calculated)
- *Register/Login* **«include»** *Authenticate user* (credentials are always verified)

«**extend**» means the extension is OPTIONAL — it adds behavior only when certain conditions hold:

- *Select insurance* «**extend**» *Make booking* (insurance is optional)
- *Apply discount code* «**extend**» *Make booking* (only if user has a code)

5. Class Diagram

The Class Diagram shows the OOP design: classes, their attributes and methods, inheritance hierarchies, and relationships. All four OOP principles are visible here.

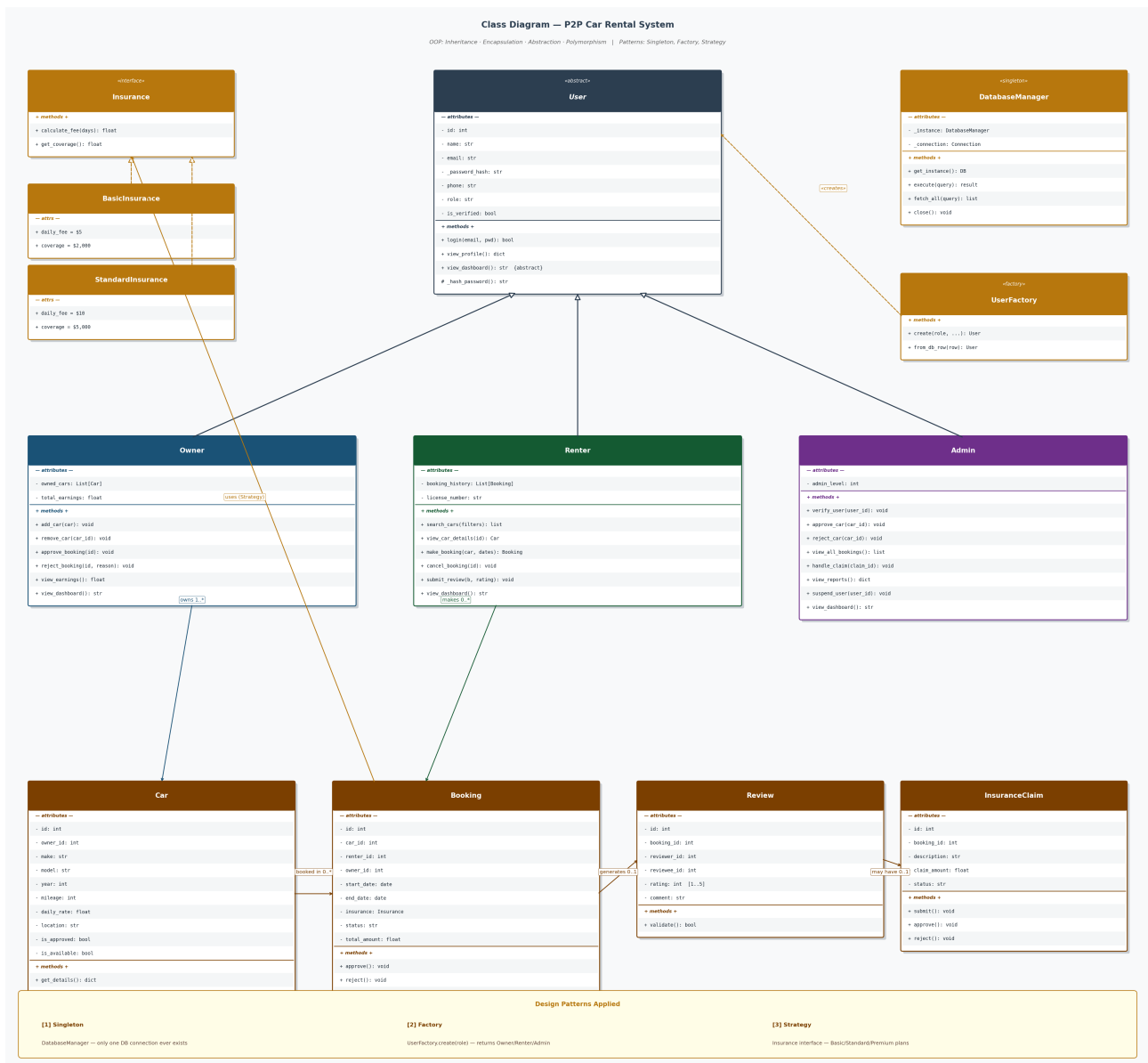


Figure 3: Class Diagram — abstract User base class, 3 role subclasses, and 3 design patterns applied.

5.1 OOP Principles in the Class Diagram

Principle	Where it appears
Encapsulation	Private attributes (e.g. <code>_password_hash</code>) accessed only via <code>@property</code> getters.
Inheritance	Owner, Renter, Admin all extend the abstract User class.
Abstraction	User is abstract; cannot be instantiated directly. <code>view_dashboard()</code> is abstract.
Polymorphism	<code>view_dashboard()</code> is overridden differently by Owner, Renter, and Admin.

6. Sequence Diagram — Make Booking

This diagram traces the most complex use case in the system — **Make Booking** — showing the chronological order of messages between actors and services. It demonstrates how the various «include» and «extend» use cases collaborate.

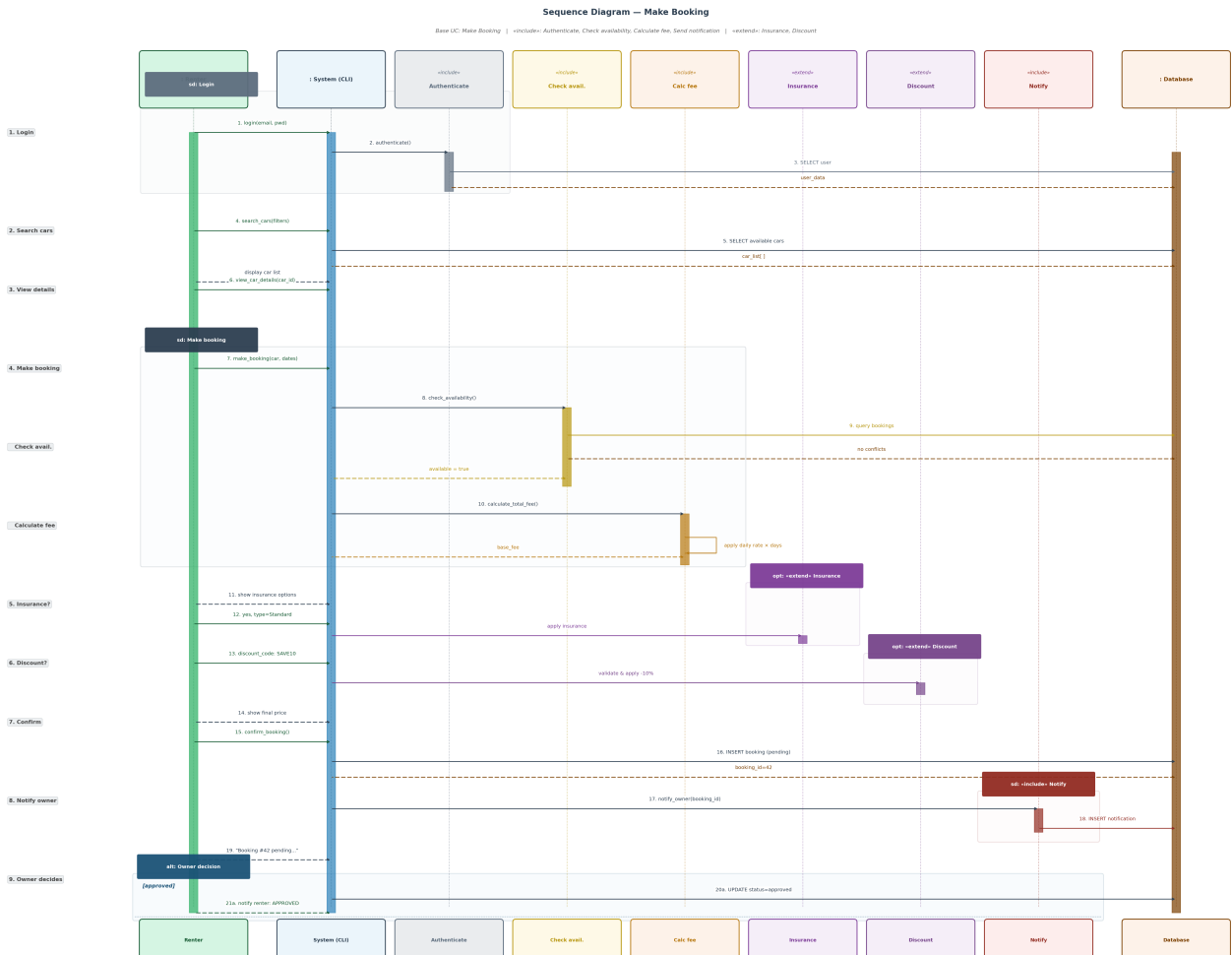


Figure 4: Sequence Diagram for the Make Booking use case, showing all participants and their interactions over time.

7. Activity Diagrams

Activity diagrams show the workflow of key processes — what decisions are made, in what order, and how parallel activities synchronize. The system has four key activity flows.

7.1 Activity: Make Booking

The Renter's end-to-end booking flow. Shows the validation loops (re-search if car not suitable), parallel includes (check availability and calculate fee happen via fork/join), and optional extensions (insurance, discount).

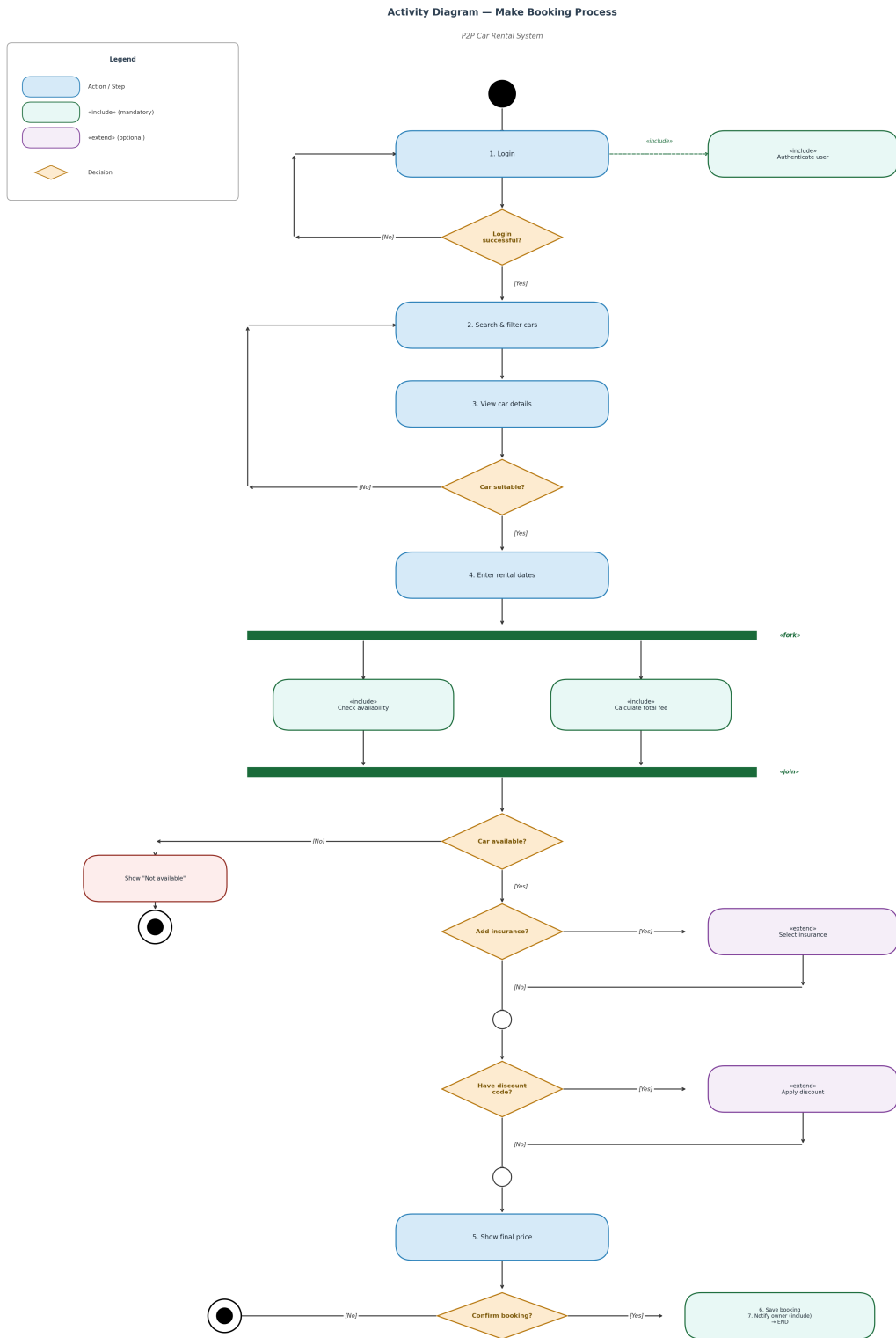


Figure 5: Activity Diagram — Make Booking process from login through owner approval.

7.2 Activity: Add Car Listing

The Owner adds a car; the Admin approves it. Two swimlanes show the handoff between roles. Validation loop catches invalid input before submitting.

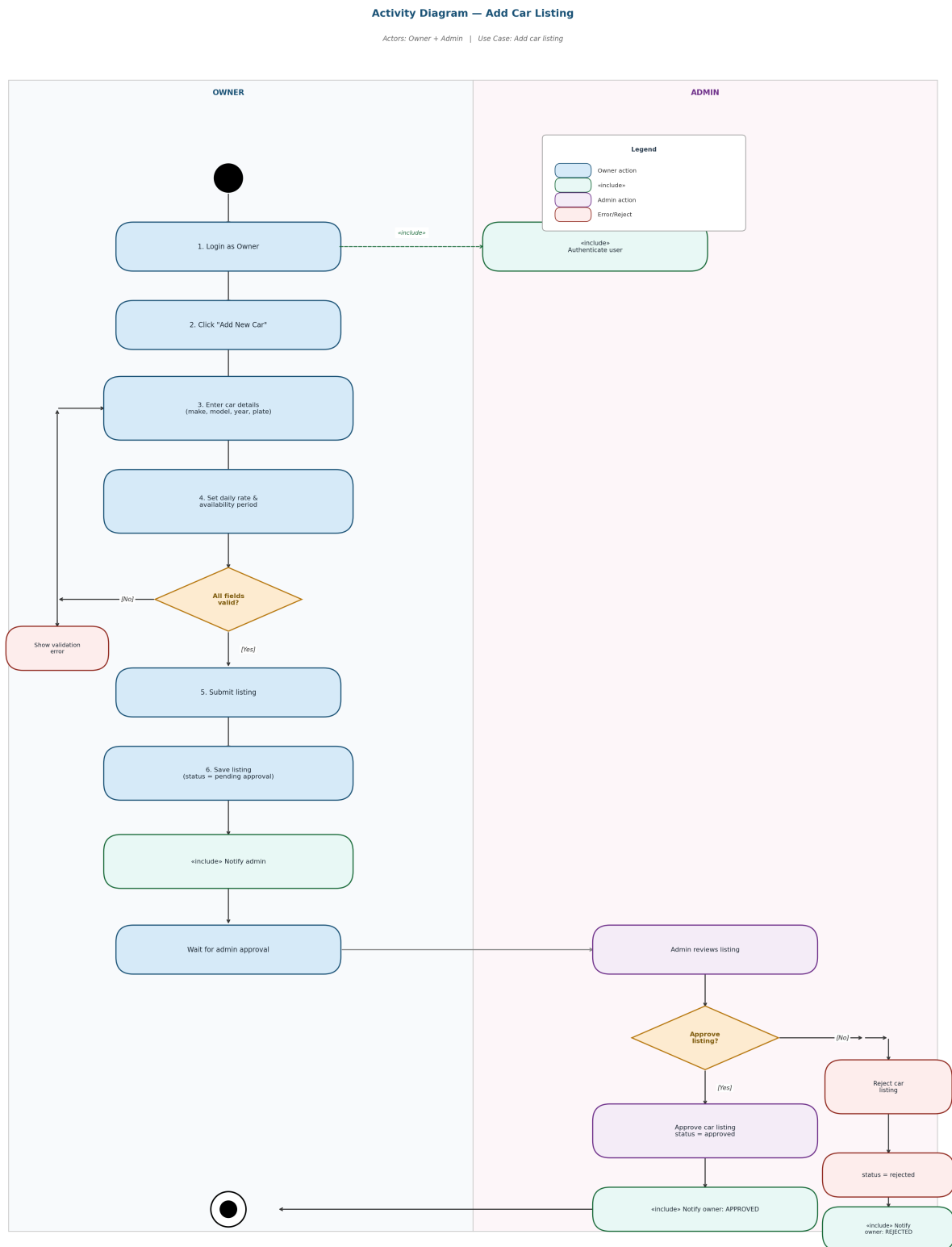


Figure 6: Activity Diagram — Add Car Listing (Owner submits → Admin approves).

7.3 Activity: Approve / Reject Booking

The Owner reviews incoming booking requests. Two decision diamonds (renter rating, car availability) guide the choice between approve and reject paths. The reject path requires a reason — a required field with validation.

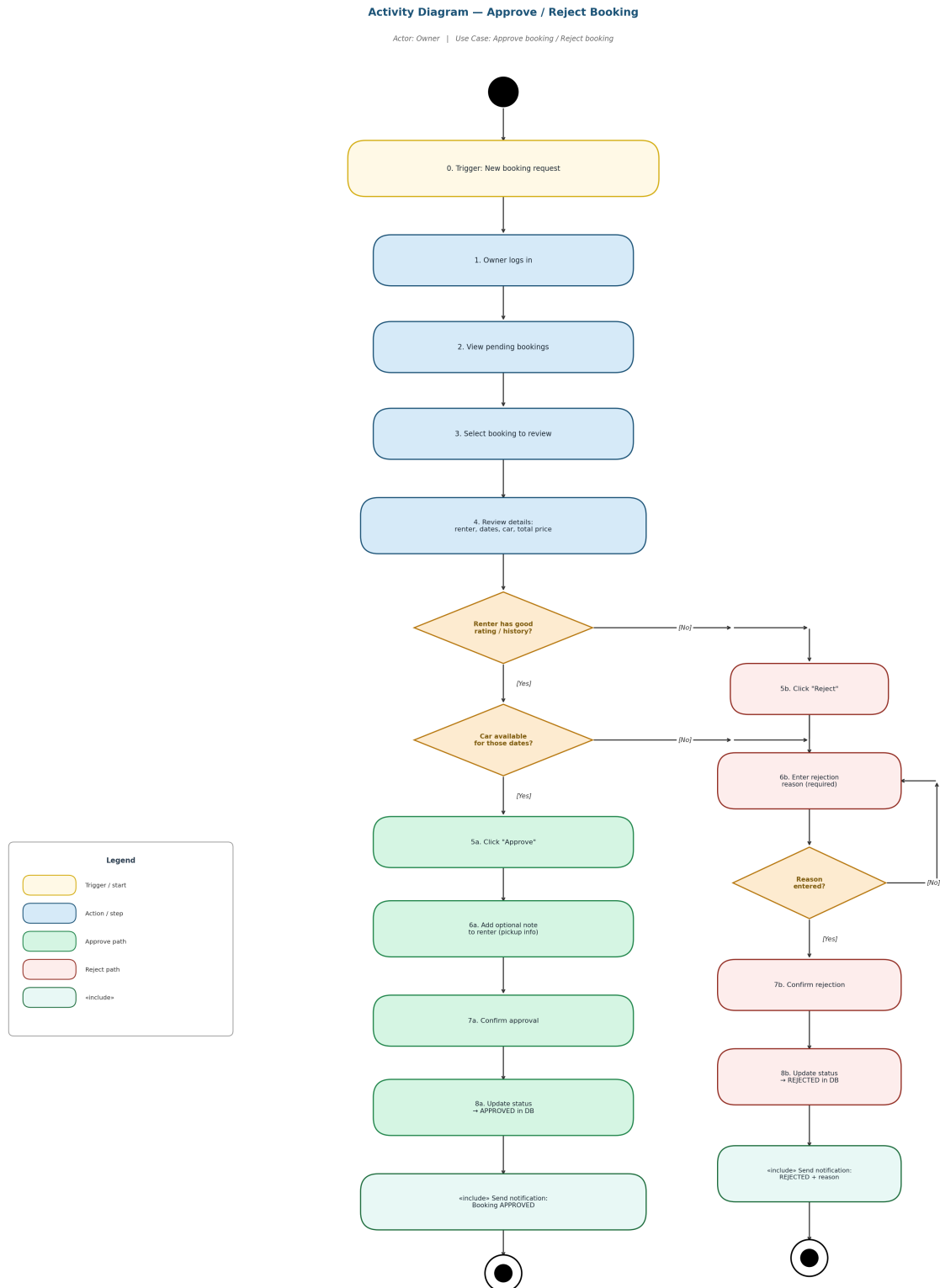


Figure 7: Activity Diagram — Owner approves or rejects a booking request.

7.4 Activity: Handle Insurance Claim

The most complex flow — three swimlanes (Owner, Admin, Renter) with fork/join for parallel evidence collection. Three possible outcomes: full approve, partial payout, or reject.

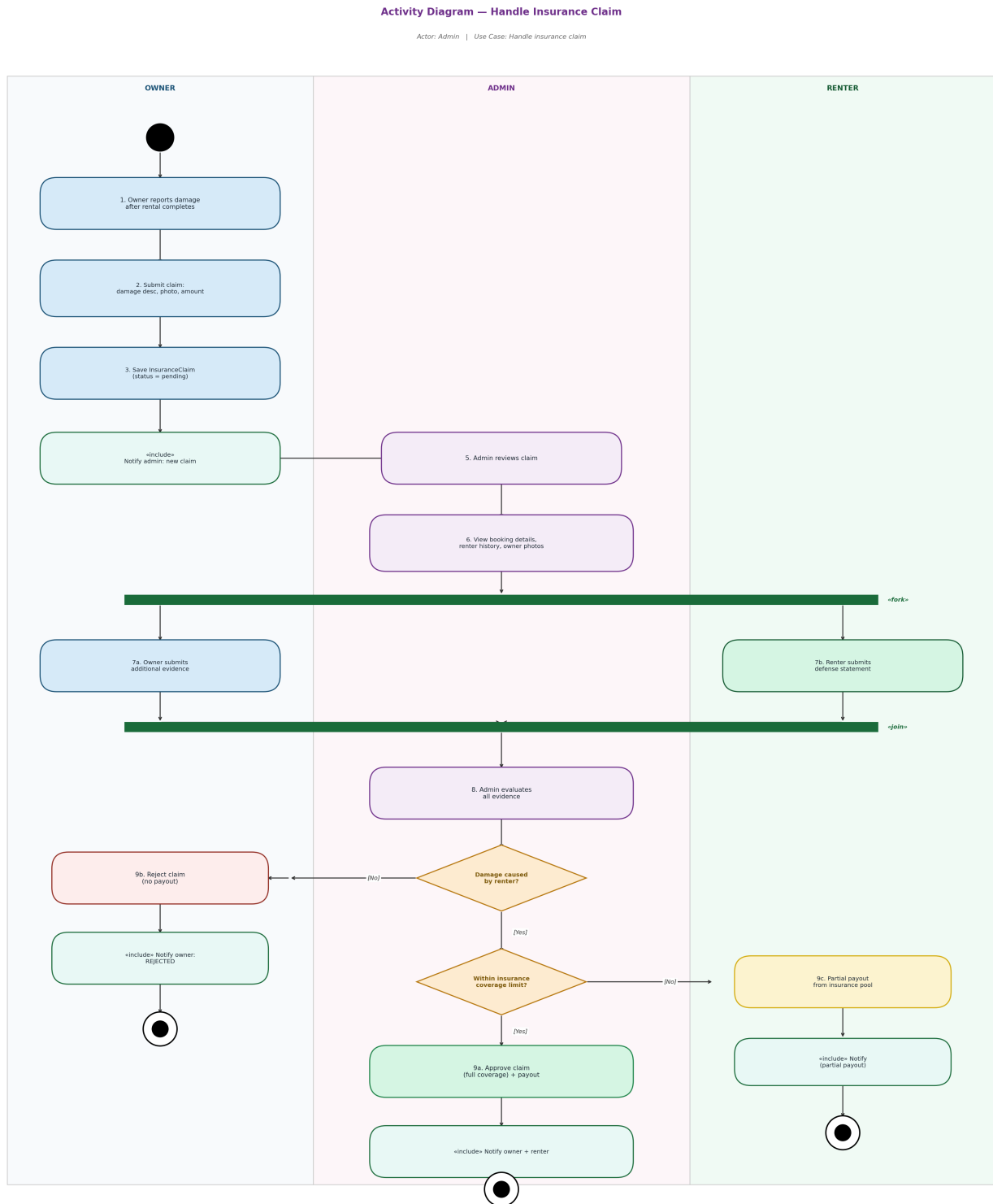


Figure 8: Activity Diagram — Handle Insurance Claim (3 swimlanes, fork/join, multi-way decisions).

8. Design Patterns Applied

The system applies **three classic design patterns** to improve modularity, maintainability, and extensibility.

8.1 Singleton Pattern — DatabaseManager

Problem: If every service opened its own database connection, we'd quickly hit connection limits or get race conditions on writes.

Solution: DatabaseManager enforces that only ONE instance ever exists. Every service that imports it gets the same connection.

```
class DatabaseManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialize()
        return cls._instance
```

Benefits: Centralized connection management; tables and indexes created once on startup; clean shutdown via close().

8.2 Factory Pattern — UserFactory

Problem: When we load a user from the database, we know their *role* (string) but need to create the right Python subclass — Owner, Renter, or Admin. Scattered if/elif chains throughout the codebase would be a maintenance nightmare.

Solution: UserFactory centralizes user creation. Anywhere we need to build a User object, we call *UserFactory.create(role, ...)* and the factory decides which subclass to instantiate.

```
class UserFactory:
    @staticmethod
    def create(role, user_id, name, email, ...):
        if role == 'owner':
            return Owner(user_id, name, email, ...)
        elif role == 'renter':
            return Renter(user_id, name, email, ...)
        elif role == 'admin':
            return Admin(user_id, name, email, ...)
```

Benefits: Adding a new role (e.g. fleet_manager) requires changes in only ONE place; no if/elif duplication across services.

8.3 Strategy Pattern — InsuranceStrategy

Problem: The system offers three insurance plans (Basic, Standard, Premium) with different fees and coverage limits. Hard-coding if/elif logic in the booking service would make adding new plans painful.

Solution: Each insurance plan is its own class implementing the InsuranceStrategy interface. The Booking object holds a reference to one Strategy and delegates fee calculation to it.

```
class InsuranceStrategy(ABC):
    @abstractmethod
    def calculate_fee(self, days): pass

class BasicInsurance(InsuranceStrategy):
    daily_fee = 5.0 # $5/day, $2000 coverage

class StandardInsurance(InsuranceStrategy):
    daily_fee = 10.0 # $10/day, $5000 coverage

class PremiumInsurance(InsuranceStrategy):
    daily_fee = 15.0 # $15/day, full coverage
```

Benefits: Adding a new insurance plan is just adding a new class. The booking service code never changes. This follows the *open/closed principle* — open for extension, closed for modification.

8.4 Patterns Summary

Pattern	Used For	File
Singleton	Database connection (one and only one)	database/db_manager.py
Factory	Creating User subclasses by role	factories/user_factory.py
Strategy	Pluggable insurance plans	patterns/insurance_strategy.py

9. Component Interactions Summary

This section summarizes how the major components interact at runtime. Understanding these interactions is essential for anyone maintaining or extending the system.

9.1 Startup Sequence

Step	Component	Action
1	main.py	Reads APP_ENV environment variable; loads config.
2	DatabaseManager	Singleton initializes: opens SQLite connection, creates tables and indexes.
3	DatabaseManager	Seeds default admin account if none exists.
4	ui/main_menu	Displays login/register menu; awaits user input.
5	AuthService	Validates credentials on login; returns User instance.
6	UserFactory	Creates the correct subclass (Owner/Renter/Admin) from DB row.
7	ui/owner_menu (or renter/admin)	Polymorphic dispatch to role-specific menu.

9.2 Make Booking Interaction Chain

Step	Caller	Callee	Purpose
1	Renter (CLI)	CarService.search_cars	Get available cars
2	CarService	DatabaseManager.fetch_all	SELECT cars
3	Renter (CLI)	BookingService.check_availability	«include» mandatory
4	Renter (CLI)	get_insurance(name)	Factory for Strategy
5	Renter (CLI)	BookingService.calculate_total	«include» mandatory
6	BookingService	InsuranceStrategy.calculate_fee	Polymorphic call
7	Renter (CLI)	BookingService.make_booking	Persist to DB
8	BookingService	DatabaseManager.execute	INSERT INTO bookings

9.3 Key Design Decisions

- **SQLite over a server database:** The assessment requires zero manual configuration. SQLite is file-based and bundled with Python — no installation or setup needed.
- **Singleton over a connection pool:** The system is single-user (one CLI session at a time), so a connection pool would add complexity without benefit. The Singleton fits the use case perfectly.
- **Strategy pattern for insurance, not a simple dict:** Could we just use a lookup table for insurance fees? Yes, but each plan has additional behavior (coverage limits, claim handling) that benefits from polymorphism.
- **Layered architecture instead of a flat structure:** A small project could put everything in one file. But maintaining clear layers from day one means the codebase scales gracefully as features are added.
- **Environment-based config:** Hard-coded paths and credentials are brittle. Reading APP_ENV at startup lets us run the same code in dev, test, and prod without modification.

Conclusion: The design and architecture of the P2P Car Rental System reflect careful application of object-oriented principles, design patterns, and layered architecture. These choices directly enable the extensibility and maintainability strategies described in the Maintenance and Support Plan.